Python.

1. Variables

Concatenation.

message_1 + ' ' + message_2

= F string.

= We can use f-string to concatenate variables.

eg

msg_1 = "I'm learning python"

msg_2 = " and it's fun"

Then we can use F-string as follows

f'{msg_1}space{msg_2}'

Outcome = "I'm learning python and it's fun".

2. List.

= Used to store multiple items in a single variable

= ~~It is immutable~~

NB = It is mutable.

= We use []

e.g

Countries = ['United states', 'India', 'China', 'Brazil']

NB = Lists can have elements of different types.

e.g string integer

= List can have duplicated elements.

= By indexing we can obtain an element by its position

= Python uses 0-based indexing.

e.g United states = 0 China = 2.

India = 1. Brazil = 3.

e.g to get united states we write the following code

Countries [0] "we get USA"

Countries [1] "We get India"

= There is also negative indexing.
= It starts from -1 to ∞
= -1 represents first elements ~~Bragg~~ irring from the last.

e.g Countries [-1] 'Brazil'
Countries [-2] 'China'
Countries [-3] 'India'
Countries [-4] 'United states'

= Slicing.
= Is a subset of the lists elements.
= Means accessing part of the list.

= It has a formula. i.e
list-name [start:stop]
NB= stop element is not included in the slice.

countries [0:3]
This gives us ['United states', 'India', 'China',]

Brazil is not included because index 3 is not included in the sliced subset.

countries [1:]
= ['India', 'China', 'Brazil']

= Adding elements to a list.
= We can use the append method
e.g countries.append('Canada')
= ['USA', 'India', 'China', 'Brazil', 'Canada']

= We can also use insert method.

NB = Insert method is efficient because it can target a specific index location.

Formula = list_name.insert(index, Name of element)

e.g countries.insert(0, 'Spain')
= ['Spain', 'USA', 'India', 'China', 'Brazil', 'Canada']

NB = We can join 2 lists using + operator.
eg Countries_2 = ['UK', 'Germany']

= Countries + Countries_2

NB

Nested Lists.

= Putting two lists inside another list.
e.g
Nested_list = [countries, countries_2]

= [['USA', 'India', 'China', 'Brazil'],
['UK', 'Germany']]

Removing elements from a list.

1. Remove method.
list_name.remove(elements you want removed)
eg Countries.remove('Spain')
= ['USA', 'India', 'China', 'Brazil']

2. Pop method → Remove element with index.
= countries.pop(-1)
= ['USA', 'India', 'China']

3. Using 'del' function.

e.g

del countries [0] → Remove first element.

= ~~['USA'~~

= ['India', 'china']

Sorting a list.

1 = Using sort method.

a) Formula = variable_name.sort( ) → Default sort from low to high.

b) Formula = list_name.sort(reverse=True) → from High to low.

Update value in a list.

We use indexing to locate the element and set it a new value.

numbers = [4, 3, 10, 7, 1, 2].

e.g numbers [0] = 1000

result = [1000, 3, 10, 7, 1, 2]

Copy of a list.

1. Slicing technique. (subset)

e.g Countries_3 = countries [:] (Whole list)

Result. = ["USA", "India", "china", "Brazil"]

2. Using copy method.

Soft copy
unchangable copy

e.g Countries_5 = countries.copy()

3. Dictionary.

- Is an ordered collection of items used to store data values.
- Contains a key and a value.

eg my_dict = {'key1': 'value', 'key2': 'value2'}

eg my_data = {'name': 'Frank', 'Age': 26}

How to get keys.

- My_data.keys()

result dict_keys(['name', 'age'])

How to get values

- My_data.values()

result dict_values(['Frank', 26])

How to get items.

My_data.items

Adding items in dictionary (key+value)

- My_data['Height'] = 1.7

My_data => {'name': 'Frank', 'Age': 26, 'Height': 1.7}

Updating a value's key.

- My_data.update({'Height': 1.9})
  - {'name': 'Frank', 'Age': 26, 'Height': 1.9}

Copy a dictionary.

new_dict=my_data.copy()

Remove items from dictionary.

1. Using pop method.

My-data.pop('name')

Returns {'Age': 25, 'Height': 1.9}

2. Using del function.

del my-data['age']

returns {'Height': 1.9}

3. Using clear function.

my-data.clear()

= Removes all elements.

IF STATEMENT.

Pseudocode.

```
If <condition>:
    <code>
elif <condition>:
    <code>
else:
    <code>
```

True

True

Both False

= Look out for indentation Press TAB on your PC

FOR LOOP.

- Python for loops are used to loop through an iterable object. and performs the same action on each ~~loop~~ entry.
- Example of iterable object is a list.

Pseudocode. _temporary (does not exist)

```
for <variable> in <iterables>:
    <code>
```

eg
```
for country in countries:
    Print (i)
```
```
for country in countries:
    Print (country)
```

result = USA
India
China
Brazil.

NB Country variable is not defined = it is new

= We can also use in enumerate function in the for loop to show the index and the value of that index.

e.g
```
for i, country in enumerate (countries):
    Print (i)
    Print (country)
```

=
```
0
United states
1
India
2
China
3
Brazil.
```

For Loop in dictionary.

```
for key, value in my-dict:
    Print (key)
    Print (value)
```
.items()

my-dict.items()

Function.

- Block of code which only runs when it is called.
- You can pass data known as parameters. into a function.

Pseudocode.

```
def function (<parameters>):
    <code>
    return <data>
```

- Every function must return something.

e.g
```
def sum_values (a, b):
    x = a + b
    return x.

Sum_values (1,3)
```

Result = 4.

Built-in functions.

variable name

1. Len( ) = count the length of iterable object.
2. max( ) = Returns the items with the highest value.
3. min( ) = Returns the items with the lowest value
4. type( ) = Returns the type of an object.
5. Range( ) = Returns a sequence of numbers

e.g range (1, 10, 2)
start finish increment.

= Mostly used in for loops.

e.g
```
for i in range (1,10,2)
    Print (i)
```
= 1, 3, 5, 7, 9

Module.

= Module → are files that contain python code.
  - Modules can have classes, functions and variables. and even runnable code.

= To get access to a module, we have to use the import function.

OS Module.

= Import os.

= Functionality of OS.

1. e.g # current working directory.

e.g os.getcwd()
You get the path of that file.

2. # Listing all files and folders in a directory.

We use the function os.listdir()

```
## Finding current directory.
Print ("Current Directory:", os.getcwd())
## List items
for item in os.listdir():
    print (item)
```

3. Helps us create a new folder.

We use os.makedirs ("name of the folder")

e.g os.makedirs ("New folder")

Introduction to Pandas & Numpy

1. Pandas.

= Used to do real-world data analysis in python.
= It allows us to clean, wrangle, visualization and more.
= like a super-charged microsoft excel. i.e most of excel task can be done in pandas.

Python/Pandas Advantage over Excel.-

= Panda can handle large data.
= Can cope with memory-intensive computation.
= Panda is good for automation even with Excel having VYA that simplifies tasks.
= Python has cross-platform capabilities.

Concepts of Pandas.

Arrays → data structures like lists.
→ Can be 1d, 2d.

= Two main data structures in Pandas are series and dataframes.
= Series → is a 1 dimensional array
= Dataframe → is a 2 dimensional array. / 2 axes Row column

column → series.

e.g

	states	Population	Postal.
0	Malindi	2040	20800
1	Mombasa	5040	32400
2	Nairobi	10000	105000

observation ← Row →

↑ Index.

↑ series

keys / features

3 features 3 observation.

- A single column should have the same type of data.

Different terminology in Excel & Pandas

Excel	Pandas.
Worksheet	Dataframe.
Column	Series.
Row heading	Index.
Row	Row
Empty cell	NaN. (Not a Number)

How to create a Pandas Dataframe

= 1D array → column
= 2D array → Dataframe. [# By use of Arrays.

A). # With the use of Numpy Library.

= To use arrays we use library named numpy.

B) # With the use of List arrays.

= e.g data = [[1,4], [2,5], [3,6]]

2 # By use of dictionary.

= You can create a dataframe by dictionary by making the key (the column name) and value (as the data)
= Value can be a list.
= 1 item (key and value) is a series (1D array)

3 # CSV file.

= CSV files - are files that can be opened in spreadsheet like Excel and is the easiest way to create a dataframe. coz we only need to read the CSV file.

1:# By use of Arrays a) (Numpy)

```
Import pandas as pd.
import numpy as np.
# Creating an array
data = np.array([[1,4],[2,5],[3,6]])
# Creating a dataframe.
df = pd.DataFrame(data, index=['row1','row2','row3'],
                  columns=['col1','col2'])

# Showing dataframe.
   df
```

Output

	col 1	col 2
row 1	1	4
row 2	2	5
row 3	3	6

B) Use of a list array.

```
# Creating a list array.
data = [[1,4],[2,5],[3,6]]
# Creating a dataframe
df = pd.DataFrame(data, index=['row1','row2','row3']
                  columns=['col1','col2'])
# Showing the dataframe.
   df.
   Output: 7→
```

2 # Creating a dataframe from a dictionary.
Using lists as value.

# lists used for this examples.
states = ['California', 'Texas', 'Florida']
population = [39,000 , 44,000 , 55,000]

# storing lists within a dictionary.
dict_states = {'States': states, 'Population': population}

# Creating a dataframe.
Pop_df = Pd.Dataframe (dict_states)

showing the dataframe.
pop_df

3 # Creating a dataframe from a CSV file.
# reading the csv file.
df = pd.read
df_exams = Pd.read_csv ('Name of file.csv')
# Show first 5 rows.
df_exams.head()
# Last 5 rows.
df_exams.tail()
# Show last n rows in a dataframe.
df_exams.head(10) df_exams.tail.
getting access to the shape attribute.
df_exams.shape e.g (1000, 8)
rows columns

# display n rows.
Pd.set_option('display.max_rows', 1000)
df_exams.

Different Attributes, Methods & Functions DataFrame has in Pandas.

1. Attributes - A value associated with an object (referenced by name using dotted expression).
   e.g df.columns
- To get to the dataframe's columns we have to use (.columns) = Attribute.

2. Function - A group of related statements that perform a specific task.
   eg Built-in function e.g max() min() len()

3. Methods - A function which is defined inside a class body.
   e.g df.head()
= When we use methods we have to use ()
compared to attribute where we do not need ()

Example

```
import pandas as pd.
df_exams = pd.read_csv('StudentsPerformance.csv')
df-exams
# Attributes
# getting shape
df_exams.shape
# index attribute.
df-exams.Index          output: Range (start=0, stop=1000, step=1)
# columns attribute.
df-exams.columns        output: name of columns
```

data types of each column
df_exams.dtypes. object = string int64 = integers

2. # Methods
# first 5 columns.
df_exams.head()
# showing summary info of the df.
df_exams.info() # can show if there is blank / empty data rows / column
can show data type # number of columns.
can show range

describing basic statistics of the dataframe.
df_exams.describe() # shows count, mean, std, min, 25%, 50%, 75%, max.

3. # Functions
# obtaining number of ~~columns~~ rows of dataframe.
len(df_exams)
# obtaining highest index of the dataframe.
max(df_exams.index)
# " lowest " " " "
min(df_exams.index)
# obtaining the data type
type(df_exams)
# rounding the values of the datasets.
round(df_exams, no. of decimal point)

selecting one column
- Most likely we are doing DS & A here.

```
import pandas as pd.
# reading csv file
df_exams = pd.read_csv('Studentsperformance.csv')
df_exams.head()
```

NB We have two options to select a column from a dataframe
1. Using square brackets.

```
# select a column with []
 df_exams['gender'] it is a series (1d array)
# Check the data type.
 type(df_exams['gender'])
# series: attributes and methods.
 df_exams['gender'].index
```

2. Using dot sign (.)

```
df_exams.gender
```

= It has deficiency since it won't understand some column names that have like 2 words e.g (math score)
= to counter this, We fill the blank space btwn 2 words with (_) e.g (math_score)

```
#
df_exams.math_score  ✓
```

= its better to use [] since changes in variable names would cause confusion.

Selecting two or more columns from a dataframe.

1. Use of [[]]

```
df_exams [['gender', 'math score']]
# checking type of the selection.
type ( df_exams [['gender', 'math score']] )   output: = Dataframe.
# selecting more columns.
df_exams [['gender', 'writing score', 'math score']]
```

NB We cannot use (.) method to select more than one column.

Adding a new column to a dataframe.

1. Using a scalar value (one single value for every row)

```
# Adding new column to dataframe.
df_exams ['language score'] = 70  # will read 70 in all the rows.
```

2. Using Arrays (Numpy)

```
# import numpy
import numpy as np.
# Create an array of 1000 elements.
language_score = np.arange (0,1000)
# Check the length
len (language_score)
```

3. Using Arrays (scores are random numbers from 1 → 100)

```
# Create random integers btwn 1 and 100.
import numpy as np
np
```

```
# Creating random integer numbers btwn 1 → 100.
int_language_score = np.random.randint(1,100, size=100
# min value inclusive and high value exclusive
min(int_language_score)
max( int_language_score)

# Assigning the random array to the dataframe.
df_exams['language score'] = int_language_score
df_exams
```

```
# creating random float number btwn 1 and 100.
np.random.uniform(1,100, size=100)
```

#A Copy

4. Using assign() method. (Adding more than 1 column)

```
import pandas as pd
#read csv file to create a dataframe.
df_exams = pd.read_csv("Students performance.csv")
df_exams.head()
#We use assign method when we want to;
    ✓ Add multiple columns in a single line of code.
    ✓ Overwrite the values of an existing column/columns.
import numpy as np
# Create random numbers for our new score columns.
score_1 = np.random.randint(1, 100, size=1000)
score_2 = np.random.randint(1, 100, size=1000)
# Make the scores into column (Id= series) using pandas
series1 = pd.Series(score_1, index = np.arange(0,1000))
series2 = pd.Series( score_2, index= np.arange(0,1000))
# Using assign() to add multiple columns.
df_exams.assign(score_1= series1, score_2 = series2)
```

check the dataframe.
df_exams

Assign method creates a copy of that dataframe with the new addition.
- So as to make the copy be the same as the original dataframe we use "=" operator.
df_exams = df_exams.assign(score_1=series1, score_2=series2)
- By doing this you would have updated the columns into the series.

5. Using insert() method.
- This method inserts a new column at a specific position.
- But can only add 1 column.

```
import numpy as np
import pandas as pd.
# create variables. and series.
Test = np.random.randint(1,100, size=1000)
series_1 = pd.Series(Test, index=np.arange(0,1000))
# Using insert() method.
formula
df_exams.insert(loc, column_name, value, allow_duplicates=False)
df_exams.insert(1,'test', series_1)
```

Math operations in DataFrame.

1. Operations in columns.

```
# Select a column and calculate total sum (sum method)
df-exams ['math score'].sum()
#count, mean, std, max & min.
df-exams['math score'].count()
df-exams ['math score'].describe()
df-exams.describe()
```

2. Operation in rows.

```
# Calculating the sum in a row.
df-exams ['math score'] + df-exams ['reading score'] +
                    df-exams ['writing score']

# Calculating the average and assigning it to a new column
df-exams ['Average'] = (df-exams ['math score'] +
    df-exams ['reading score'] + df-exams ['writing score'])

df-exams.round(1)  # rounds all the average values
                   to 1 decimal place.
```

Value counts in DataFrame.

```
# Counting gender elements by category (female, male)
df-exams ['gender'].value-counts()
Output: female 518
        male 482
        Name: gender dtype int64
# Calculate percentage (relative frequence)
df-exams ['gender'].value_counts(normalize=True)
output  Female 0.518
        Male   0.482
        Name: gender dtype: float64.
```

Sorting a dataframe.
we use (.sort_values()) method.
Formula.
df_exams.sort_values([column names], ascending=True)
NB # sort_values() method makes a copy.
# Sorting 1 column.
df_exams.sort_values('math score')
# Sorting multiple values
df_exams.sort_values(['math score', 'writing score'] ascending
=True)
NB (sort_values) method only creates a copy.
total formula for sort_values.
df_exams.sort_values(
by,
axis=0
ascending=True,
Inplace=False,
Kind='quicksort'
na_position='last'
ignore_index=False,
key: 'ValueKeyFunc'=None,)

# sort descending by multiple columns and update dataframe.
df=exams = df_exams.sort_values(['mathscore', 'writing score']
, ascending=False,
inplace=True)

sorting a dataframe with a key function.
df_exams.sort_values('raceethnicity', ascending=True,
key=lambda, col: col.str.lower())

1. Create Index.

```
import numpy as np
import random.

#creating non-repetitive values for the index
new_index = np.arange(0,1000)

#shuffling the index (.shuffle() method)
random.shuffle(new_index)
#creating a new column.
df_exams['new_index'] = new_index
#set new_index column as index (.set_index method)
df_exams.set_index('new_index', inplace = True)
#sorting the index
# sort data.frame by new index. (sort_index(asc = True))
df_exams.sort_index()
or df_exams.sort_index(inplace = True)
```

2. Rename Column.

```
#rename column and overwrite values of the dataframe.
df_exams = df_exams.rename(columns = {'gender' : 'Gender'})
                                      Previous   new_column name.

#  "  "  multiple.  and update the dataframe with inplace argument
df_exams.rename(columns {'mathscore' : 'Ms',
                         'writing score' : 'Ws',
                         'reading score' : 'Rs'},
                         inplace = True)
```

3. Rename Index

```
# rename index 0, 1 and 2 and update the dataframe.
df_exams.rename(index={0:'A', 1:'B', 2:'C'},
                inplace=True)
```

NB. shift + Tab to check what a method entails.

Project 1 # Web Scraping in Pandas

= Mainly uses libraries such as = Beautiful soup
- Selenium
= Scrapy.

~~#~~ start.

```
import pandas as pd.
```

#reading a csv from Url (website) using pandas.

= Web scraping consists in extracting data from websites.
= Most people do it manually, but we can automate it with some web scraping techniques.
= Target website https://www.football-data.co.uk/data.php
= First enter the site, press on the file you want then copy the link Address and paste it into search ~~csv pd.read_csv~~ bar of the chrome. and press Enter
= This downloads the file.
= which means the link contains the data we want to scrap
= Copy
= Copy link Address to make sure

#reading it from the website.
df_premier21 = pd.read_csv('the link')
df_premier21
#rename columns.
df_premier21.rename(columns={'FTHG':'home_goals',
'FTAG':'away_goals'},
inplace=True)

Read.csv from Multiple URLs with Pandas.
Link root + season + league.
#link structure.
let's say you have this link.
https://www.football-data.co.uk/mmz4281/2122/E0.csv
this you can concatenate it this way [exactly the same]
"https://www.football-data.co.uk/mmz4281/" + "2122" +
"/" + "E0" + ".csv"

=First is the root
= Second=season
= Third = League.
= Fourth = .csv
= Instead of extracting one at a time, we are going to create a variable. so we can dynamically extract all this data.
= Instead extracting one link, we are going to extract all the links in a for-loop.

For one season. Using lists.

```
# create a root variable. (static does not change)
root = "https://www.football-data.co.uk/mm24281/"
# creating lists of leagues.
Leagues = ['E0', 'E1', 'E2', 'E3', 'EC']
 frames = [] (# to store the dataframes inside this list)
# looping through leagues, read multiple csv and append into list
  for league in leagues:        year
(Tab) df= pd.read_csv(root + '2122' + '/' + league + '.csv')
      frames.append(df)

# This is to prevent loose of data in each loop. (frames) using
# if you get a (unicode problem (invalid start byte) we
 solve it by using this built-in function.

for league in Leagues:
    df= pd.read_csv(root + '2122' + '/' + league + '.csv',
                    encoding = 'unicode_escape')
    frames.append(df)

#frames length
  len(frames)
# to check each df. E.g 'E0' first element.
  frames[0]
# last element.
  frames[4]
```

For two seasons / more.

```
# creating root.
root = "https://www.football-data.co.uk/mmz4281/"
# Concatenate the seasons e.g 2122 or 1516 (for loop)
for season in range (15,21):
    Print (str(season) + str(season+1))

# Create list of leagues and storage for the dataframes.
leagues = ['E0', 'E1', 'E2']
frames = []

# loop through seasons and insert a new column (year)
for league in leagues:
    for season in range (15,21):
        df = Pd.read_csv (root + str(season) + str(season+1)
             + '/' + league + '.csv')
        df.insert (1, 'season', season)
        frames.append (df)

# length of the frames. =18
len (frames)
# show 1st, 2nd & last df.
frames [0]
frames [1]
frames [17]

# For two/more seasons using dictionary.

# create a dictionary with original name of league as
key.
```

```
dict_countries = {
                 'Spanish La Liga': 'SP1',
                 'German Bundesliga': 'D1'
                 'English Premier League': 'E0' }

#create a dictionary that will hold all the dataframes.
 dict_historical_data = { }   #empty dictionary.

# create a for loop in dictionary that is going to
  get the values of the keys.
  for League in dict_countries:
        print (dict_countries[league]) → #Value

# this #value we will use to concatenate in the
  web_address (instead of leagues ⇒ dict_countries[league]

# looping through key elements.
  for league in dict_countries:                    (two leagues)
        frames [] - inside loop to prevent mixing of dataframes.
        for season in range (15,21):   loop through the seasons-
loop →  (Tab) df = pd.read_csv (root + str(season)+str(season+1)+
                                '/' + dict_countries [League] + '.csv')
        →  Tab(space)
Adding column to the df at Pos 2 → df.insert (1, 'season', f"{season}/{season+1}")
temporarily stores the list dataframe → frames.append (df)
        (two Tab) df_concat = pd.concat (frames)   to concatenate every seasons of a certain league.
saving the whole dataframes (no Tab) dict_historical_data [League] = df_concat

# show dataframe e.g English Premier League
  dict_historical_data ['English Premier League']
```

Filtering dataframes in Pandas.

1# Filter dataframe based on a condition.

```
import pandas as pd
df_laptops = pd.read_csv(r"laptop_price.csv)
df_laptops.head(3)
```

Comparator <

filtering based on one condition (operator comparison)

which rows have "apple" in the "company" column.

dataframe ['column'] == 'string'

e.g df_laptops ['company'] == "Apple"

NB You will get values of True and False.

Filter the dataframe. (Smaller dataframe)

Dataframe [comparison]

- df_laptops [df_laptops ["company"] == "Apple"]

NB it will return a dataframe that has apple only in the company column name.

= For bigger dataframe we use (.value_counts())

Formula = filtered dataframe.value_counts ('column name')

= df_laptops [df_laptops ['Company'] == "Apple"].value_counts ('company')

= It will return the number of Apple in that company column.

Exercise 1.

Find which rows don't have "HP" in the 'company' column.

=# filter dataframe based on that condition.

Comparison = df_laptops ["company"] != "HP"

Filter 2 ways

smaller dataframe.

df_laptops [df_laptops ["company"] != "Hp"]

bigger dataframe
df_laptops [df_laptops ['company'] != "Apple"].value_counts ('company')

Exercise 2
Find laptops with price over 2000 euros.
df_laptops ["Price_euros"] > 2000
Comparison filtering
df_laptops [df_laptops["Price_euros"] > 2000].value_counts ("price_euros")

Creating a column based on 1 condition.
(np.where())
import numpy as np.

= We are going to create price tiers based on price_euros column. (new_column)
comparison and np.where
1 # create an array based on price tiers.
np.where (df_laptops ['price_euros'] > 2000, 'Expensive', 'cheap')
True False

NB np.where(comparison, replacement of True, replacement of False)

2. # set array to a new column.
df_laptops ["Price_tier"] = np.where (df_laptops ['Price_euros'] > 2000, "Expensive", "Cheap"]

#Show first 5 rows.
df_laptops.head()
# count values in price_tier column.
df_laptops.value_counts ('price_tier') # (Cheap 1166, Exp 137)

Exercise.

Create an array based on screen size
Set it to a new column.
show first 5 rows.
count values in screen_size column.

NB: Column name = inches.
comparison = (>15 = big 15< = small).

# creating an array using np.where.
import numpy as np.

if inches column dtype=int

np.where(df_laptops['inches']>15, 'Big', 'Small')

setting it to a new column.

df_laptops['screen_size'] = np.where(df_laptops['inches']>15,
'Big', 'Small')

df_laptops.head(5)
df_laptops.value_counts('screen_size')

Filter dataframe based on multiple conditions. (using (and) and (or) operators) and → (&) or → (|)

Formula 1 comparison.

(condition 1) and/or (condition 2)

2. Filtering.

dataframe[(condition 1) and/or (condition 2)]

AND OPERATOR.

Comparison: ("Apple laptops costing more than 2000 euros)

(df_laptops['company']=='Apple') & (df_laptops['Price_euros']>2000)

Filtering.

df_laptops[(df_laptops['company']=='Apple') & (df_laptops['Price_euros']>2000)]

operator or(|)
Find apple or Dell laptops.
comparison (df-laptop['company']=="Apple") | (df-laptop['company']=='De
filter df-laptops [(df-laptop['company']=='Apple') | (df-laptop
['company']=='Dell')]

Find laptops from Apple or Dell costing more than 2000
((df-laptop['company']=='Apple') | (df-laptop['company']='Dell'))
& (df-laptop['price-euros']>2000)

Creating a conditional column from more than 2 choices (np.select) (same as np-where)
15, import numpy as np.
# create two variables 1. condition 2. values.
conditions = [
df-laptops['Price-~~Rank~~']>3000,
(df-laptops['Price_euros']>2000) & (df-laptop['Price_euros']<=3000),
(df-laptops['Price_euros']>800) & (df-laptop['Price_euros']<=2000),
df-laptops['Price-euros']<=800.
]
values = ['Too Expensive', 'Expensive', 'Affordable', 'Cheap']

set it to a new column.
df-laptops['Price-Rank'] = np.select(conditions, values)

df-laptops.head()
df-laptops[~~'Price-Rank'~~].value-counts('Price-Rank')
or df-laptop['Price-Rank'].value_counts()

Duplicates in Python.

finding duplicates in one column

formula = datasetname.duplicated("name of the column")

df_laptops.duplicated('Laptop_ID')

filtering the dataset:

df_laptops[df_laptops.duplicated('Laptop_ID')]

will show empty datasets if there are no duplicates

finding duplicates in more than one column.

x = df_laptops.duplicated(['Product', 'TypeName', 'Inches'])

Show all the values duplicated (except for the first → keep='first') By default.

~~df~~

~~df = laptops[df_la~~

df_laptops[x].sort_values(['Product', 'TypeName'])

Example.

Find the cheapest and most expensive laptop of each company using the (sort_value & duplicated) methods.

CHEAPEST

1 # Sort df ascending by company and price.

df_laptops = df_laptops.sort_values(["Company", "Price_euros"])

2 # Check all value categories in "Company" column.

df_laptops.value_counts("Company") to check if all values are duplicated

3 # Find the duplicated values inside the "Company" column.

duplicated_first = df_laptops.duplicated("Company", keep='first')

Filter the dataframe

df_laptops[duplicated_first]

Show dataframe with non-duplicated values in the 'company' column (~)

```
= df_laptops[~df_laptops.duplicated(subset='Company')]
```

None are duplicated in this filtered dataframe.

NB Since we sorted the dataframe from cheap to expensive and we are looking for non-duplicated strings in the 'company' column. The first one (which is cheapest) for each company will remain only. Hence when we select dataframe of 'company' and 'price' column we will get the cheapest of each company.

⇒ `df_laptops[~df_laptops.duplicated(subset='Company')][['Company', 'Price_euros']]`

Confirm if there are no duplicated.

`df_laptops[~df_laptops.duplicated(subset='Company')].value_counts('Company')`

MOST EXPENSIVE.. (keep='last')

```
# sort the dataframe.
df_laptops.sort_values(["Company", "Price-euros"])

# check for duplicates.
df_laptops.duplicated(['Company', 'Price
# check for duplicates.
duplicated=df_laptops.duplicated("Company", keep='last')
- last

#Filter the dataframe.(~) dataframe with non-duplicates
df_laptops[~duplicated_last][['Company','Price-euros']]
```

Keep='False' → will return empty if there are duplicates in the column

Dropping duplicated elements with the [drop_duplicated()] method instead of (~)

```
df_laptops.sort_values(["Company", "Price-euros"])
# in 2 or more columns.
df_laptops.drop_duplicates(["Company"])
```

→ keep='first' (Automatic) we get the cheapest one.

```
#for expensive (It is a copy)
df_laptops.drop_duplicates(["Company"], keep='last')
```

we get the most expensive laptop for each company

```
# ignore_index='False' original
```

→ if we make it True it will start from 0 → laptops.

Exercise.
Find the biggest and smallest screen size in laptops of each company.

```
df-laptops.sort_values(["Company", "Inches"])
    #Smallest screen size.
dupt = df-laptops.drop_duplicates(["Company"], keep='first')
                                  [["Company", "Inches"]]
    # Biggest screen size.
df-laptops.drop_duplicates(["Company"], keep='last')
                                  [["Company", "Inches"]]
```

Unique method.
- Returns unique values of series (column) object.

e.g # get unique elements in the company column.

```
df-laptops["Company"].unique()
```

#We get array that return unique companies.

get the length of unique element.

```
len(df-laptops["Company"].unique())
    18
```

nunique method.

→ Returns number of unique elements in the series object.

```
df-laptops["Company"].nunique()
    = 18
```

Data extraction with pandas.

1. Selecting data with loc and iloc methods

1. loc method

- it is label-based.
- We have to specify rows and columns based on their index/row and column labels.

formula = loc[name of index, name of column]

2. iloc method()

- it is an integer position based.
- We have to specify rows and columns by their integer position values (0-based integer position)

iloc[start, stop]

syntax

to get a value of a row or column

df.loc['zero'] df.iloc[0]

to get a list

df.loc[['zero','two']] df.iloc[[0,2]]

slicing a dataset.

df.loc['zero','two'] df.iloc[0:2]

('zero' and 'two' included; 0 included, 2 excluded.)

```
import pandas as pd
df = pd.read_csv(r'players_20.csv')
# set index to short name of the players.
df = df.set_index("short_name")
```

```
#select columns to work with.
df= df [["long_name", "age", "dob", "height-cm", "weight-kg"
         "nationality", "club" ]]
```

1. #selecting with a single value. using loc()
```
#get all data about L.Messi
  df.loc ['L.Messi']
#get height. of messi
  df.loc ['L.Messi', 'height_cm']
#get all rows inside the "height-cm' column.
  df.loc [:, "height-cm"] #Returns index and heights.
```

2. #get all data about Messi & Ronaldo.
#selecting elements with a list of values.
```
x = ['L.Messi', 'Cristiano Ronaldo']
  df.loc [x, :]
#get the height of messi and Ronaldo.
  df.loc [x, "height-cm"]
#get the height and weight of L.Messi.
  y = ["height-cm", "weight-kg"]
  df.loc ['L.Messi', y]
#get the height and weight of Messi and Ronaldo.
  df.loc [x, y]
```

3. #selecting a range of data with a slice.
for loc both slicing both start & stop are included.
```
x= ['L.Messi', 'Cristiano Ronaldo']
  df.loc [x, "age":"club"]  [Rows, sliced columns]
#slicing rows (top 10 players)
 df.index [:10] the first 10 indexes Messi 1st Salah 10th.
  df.loc ['L.Messi':'M.Salah', columns] #[sliced Rows, columns]
  columns = ['age', 'dob', 'height_cm', 'weight_kg']
```

4. #selecting elements based on a condition. # One Condition.

eg df['Height-cm'] > 180

columns = ['age', 'dob', 'height-cm', 'weight-kg']

df.loc[df['height-cm']>180, columns]

df.loc[argument, selection of columns u want]

#'selecting elements based on multiple condition.

x = ((df['Height-cm']>180) & (df['Nationality'] == 'Argentina'))

df.loc[x,:] # Players with height above 180cm from Argentina.

Values

1. Data Extraction with iloc[] method.

get the height of L.Messi.

df.iloc[0,3]

weight of ronaldo.

df.iloc[1,4]

all rows inside "height-cm" column.

df.iloc[:,3]

get all columns on L.Messi.

df.iloc[0,:]

2. With a list of values.

get all data about L.Messi and Cristiano Ronaldo.

~~x=[0,1]~~

df.iloc[[0,1]]

get the height of Messi & Ronaldo.

df.iloc[[0,1],3]

get the height and weight of Messi & Ronaldo.

df.iloc[[0,1],[3,4]]

3. # selecting a range of data with a slice

players = [0, 1]

columns from age → club.

df.iloc [players, 1:7]

selecting top 10 and columns from age to club.

~~df.iloc [[0,10], 1:7~~

df.iloc [0:10, 1:7]